

Modèles et tâches d'encodeur

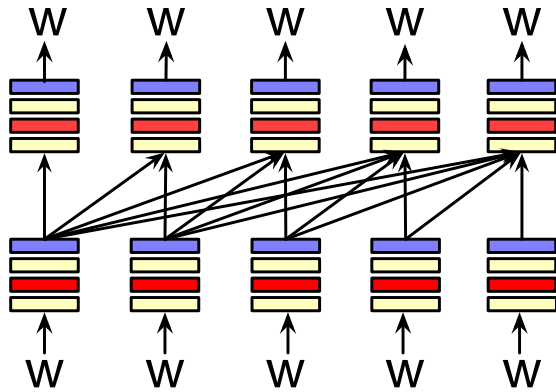
TALN Semaine 9

Merci à Dan Jurafsky et Marius Mosbach pour la plupart des diapos cette semaine !

Plan pour aujourd'hui

1. Modèles d'encodeur
2. Modélisation du langage masqué
3. Classification avec encodeurs
4. Recherche d'informations avec des encodeurs
5. *Exercices de groupe: étude de cas*

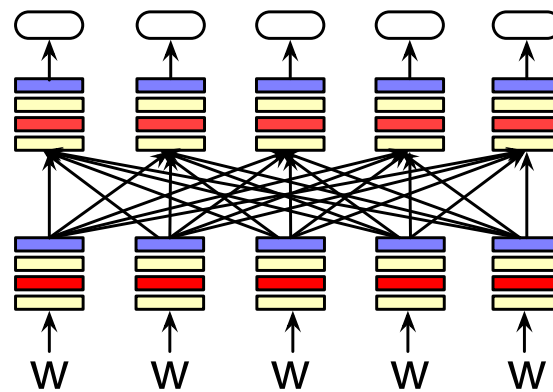
Trois architectures pour les grands modèles de langage



Decodeurs

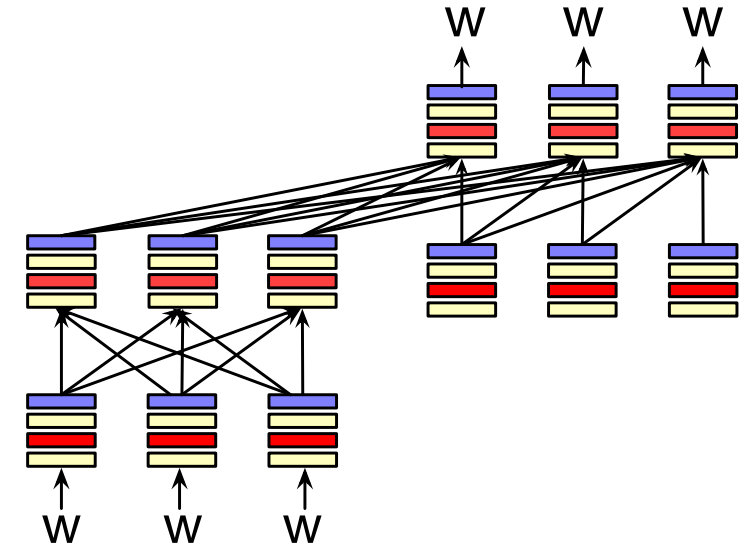
GPT, Claude,
Llama

Mixtral



Encodeurs

BERT family,
HuBERT

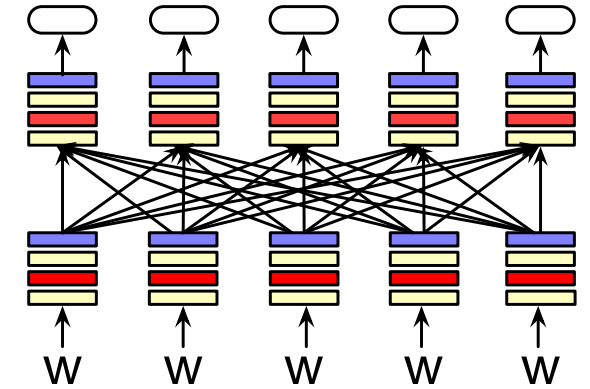


Encodeur-décodeurs

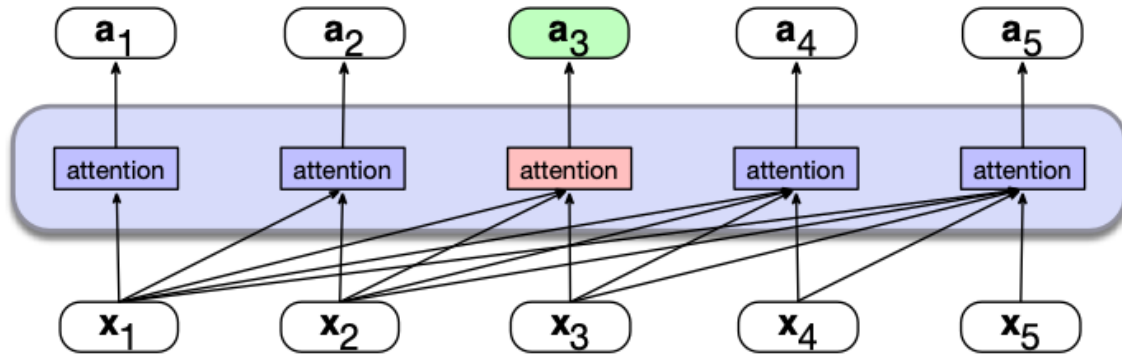
Flan-T5, Whisper

Encodeurs

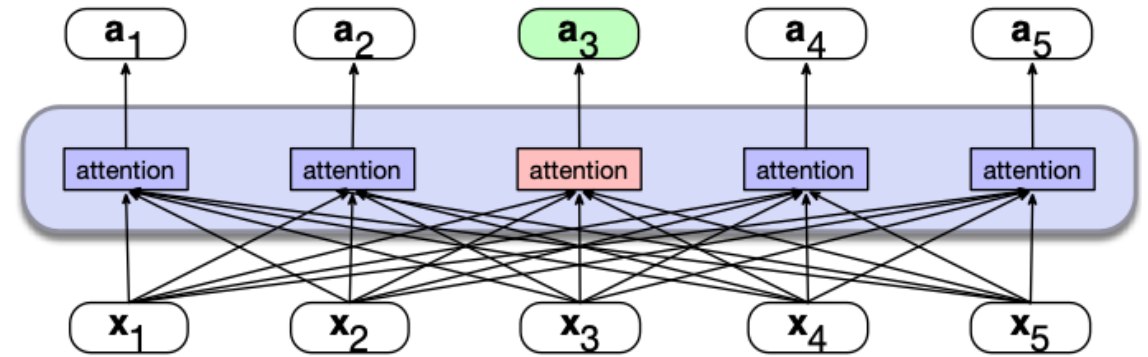
- Masked Language Models (MLMs)
- Models BERT
- Formé en prédisant les mots à partir des mots environnants des deux côtés (bi-directionnel)
- Sont généralement **peaufinés** (sur des données supervisées) pour les tâches de classification.



Attention bidirectionnel



a) A causal self-attention layer



b) A bidirectional self-attention layer

On enlève simplement le masque

Self-attention causal

N

$q1 \cdot k1$	$-\infty$	$-\infty$	$-\infty$
$q2 \cdot k1$	$q2 \cdot k2$	$-\infty$	$-\infty$
$q3 \cdot k1$	$q3 \cdot k2$	$q3 \cdot k3$	$-\infty$
$q4 \cdot k1$	$q4 \cdot k2$	$q4 \cdot k3$	$q4 \cdot k4$

N

$$\mathbf{A} = \text{softmax} \left(\mathbb{M} \left(\frac{\mathbf{QK}^T}{\sqrt{d^k}} \right) \right)$$

$$\mathbf{A} = \text{softmax} \left(\frac{\mathbf{QK}^T}{\sqrt{d^k}} \right)$$

On enlève simplement le masque

Self-attention causal

N

$q1 \cdot k1$	$-\infty$	$-\infty$	$-\infty$
$q2 \cdot k1$	$q2 \cdot k2$	$-\infty$	$-\infty$
$q3 \cdot k1$	$q3 \cdot k2$	$q3 \cdot k3$	$-\infty$
$q4 \cdot k1$	$q4 \cdot k2$	$q4 \cdot k3$	$q4 \cdot k4$

N

$$\mathbf{A} = \text{softmax} \left(\mathbb{M} \left(\frac{\mathbf{QK}^T}{\sqrt{d^k}} \right) \right)$$

Self-attention bidirectionnel

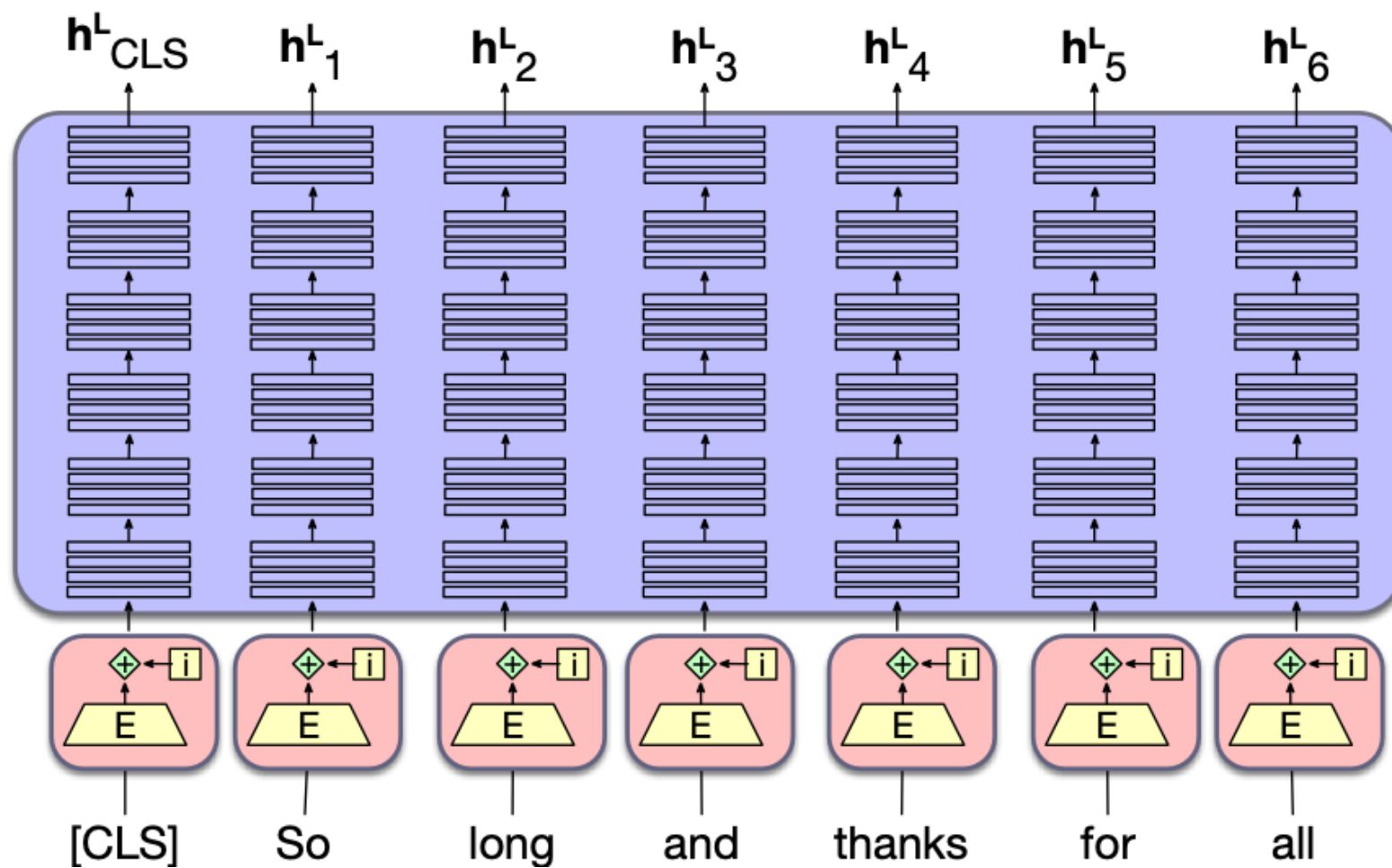
N

$q1 \cdot k1$	$q1 \cdot k2$	$q1 \cdot k3$	$q1 \cdot k4$
$q2 \cdot k1$	$q2 \cdot k2$	$q2 \cdot k3$	$q2 \cdot k4$
$q3 \cdot k1$	$q3 \cdot k2$	$q3 \cdot k3$	$q3 \cdot k4$
$q4 \cdot k1$	$q4 \cdot k2$	$q4 \cdot k3$	$q4 \cdot k4$

N

$$\mathbf{A} = \text{softmax} \left(\frac{\mathbf{QK}^T}{\sqrt{d^k}} \right)$$

BERT



Bidirectional Encoder Representations from Transformers

BERT (Devlin et al., 2019)

- 30,000 tokens en anglais seulement (WordPiece tokenizer)
- Fenêtre de contexte $N=512$ tokens, et dimension du modèle $d=768$
- $L=12$ couches de transformer blocks, chacun avec $A=12$ couches d'attention multi-tête bidirectionnelles.
- Environ 100M paramètres au total.

XLNet-RoBERTa (Conneau et al., 2020)

- 250,000 tokens multilingue (SentencePiece Unigram LM tokenizer)
- Fenêtre de contexte $N=512$ tokens, dimension du modèle $d=1024$
- $L=24$ couches de transformer blocks, chacun avec $A=16$ couches d'attention multi-tête bidirectionnelles.
- Environ 550M paramètres au total.

Modélisation du langage masqué (MLM)

- Nous avons vu des LM autorégressifs (de gauche à droite).
- Mais qu'en est-il des tâches pour lesquelles nous voulons voir les futurs tokens ?
- Particulièrement vrai pour les tâches où nous mappons chaque token d'entrée à un token de sortie (classification séquentielle)
- **Les encodeurs bidirectionnels utilisent l'auto-attention non masquée pour:**
 - Partir des embeddings à l'input $\mathbf{x}_1, \dots, \mathbf{x}_n$
 - Au embeddings de sortie de même longueur $\mathbf{h}_1, \dots, \mathbf{h}_n$
 - Où les vecteurs de sortie ont été contextualisés en utilisant des informations de l'ensemble de la séquence d'entrée.

Intuition d'entraînement masquée

- Pour les LM de gauche à droite (décodeur), le modèle essaie de prédire le dernier mot à partir des mots précédents :

The water of Walden Pond is so beautifully _____

- Pour les **LM masqués bidirectionnels**, le modèle essaie de prédire un ou plusieurs mots manquants à partir de tous les autres mots :

The _____ of Walden Pond _____ so beautifully blue

- Le modèle génère une distribution de probabilité sur le vocabulaire pour chaque token manquant
- Nous utilisons la perte d'entropie croisée de chacune des prédictions du modèle pour le processus d'apprentissage.

Entraînement MLM de BERT

15% des tokens sont choisis au hasard pour être masqués

Exemple: "Lunch was **delicious**", Si *delicious* a été choisi au hasard:

Trois possibilités pour ce 15%:

1. 80%: tokens sont remplacés par un token spécial [MASK]

Lunch was **delicious** -> Lunch was [MASK]

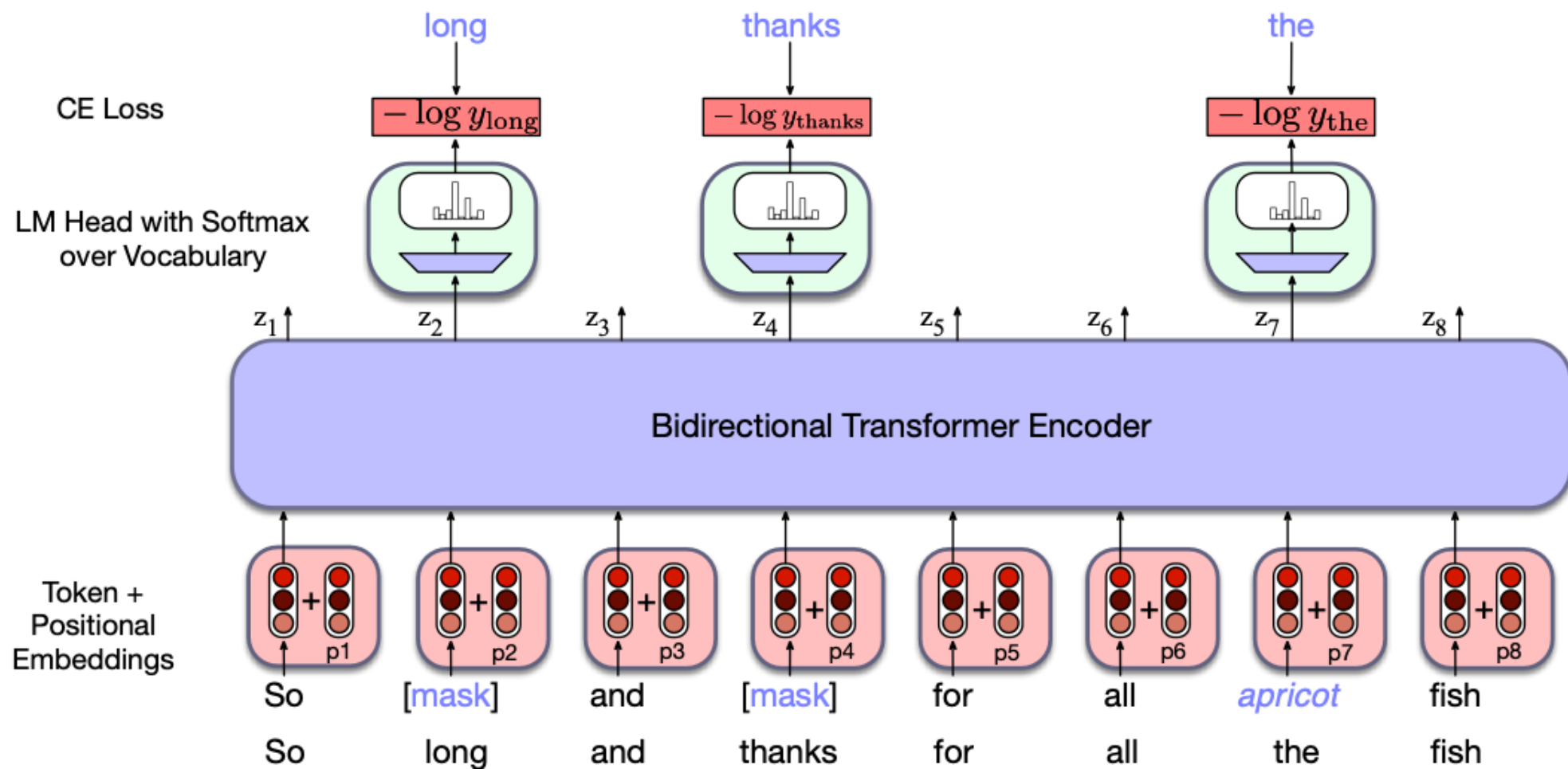
2. 10%: tokens sont remplacé par un token aléatoire (échantillonné à partir d'un model unigram)

Lunch was **delicious** -> Lunch was **gasp**

3. 10%: Tokens sont inchangés

Lunch was **delicious** -> Lunch was **delicious**

En détail



Fonction de perte MLM

La tête de modélisation du langage prend la sortie de la couche finale du transformer L, la multiplie par la couche unembedding et les transforme et probabilités:

$$\mathbf{u}_i = E\mathbf{h}_i^L \qquad \mathbf{y}_i = \text{softmax}(\mathbf{u}_i)$$

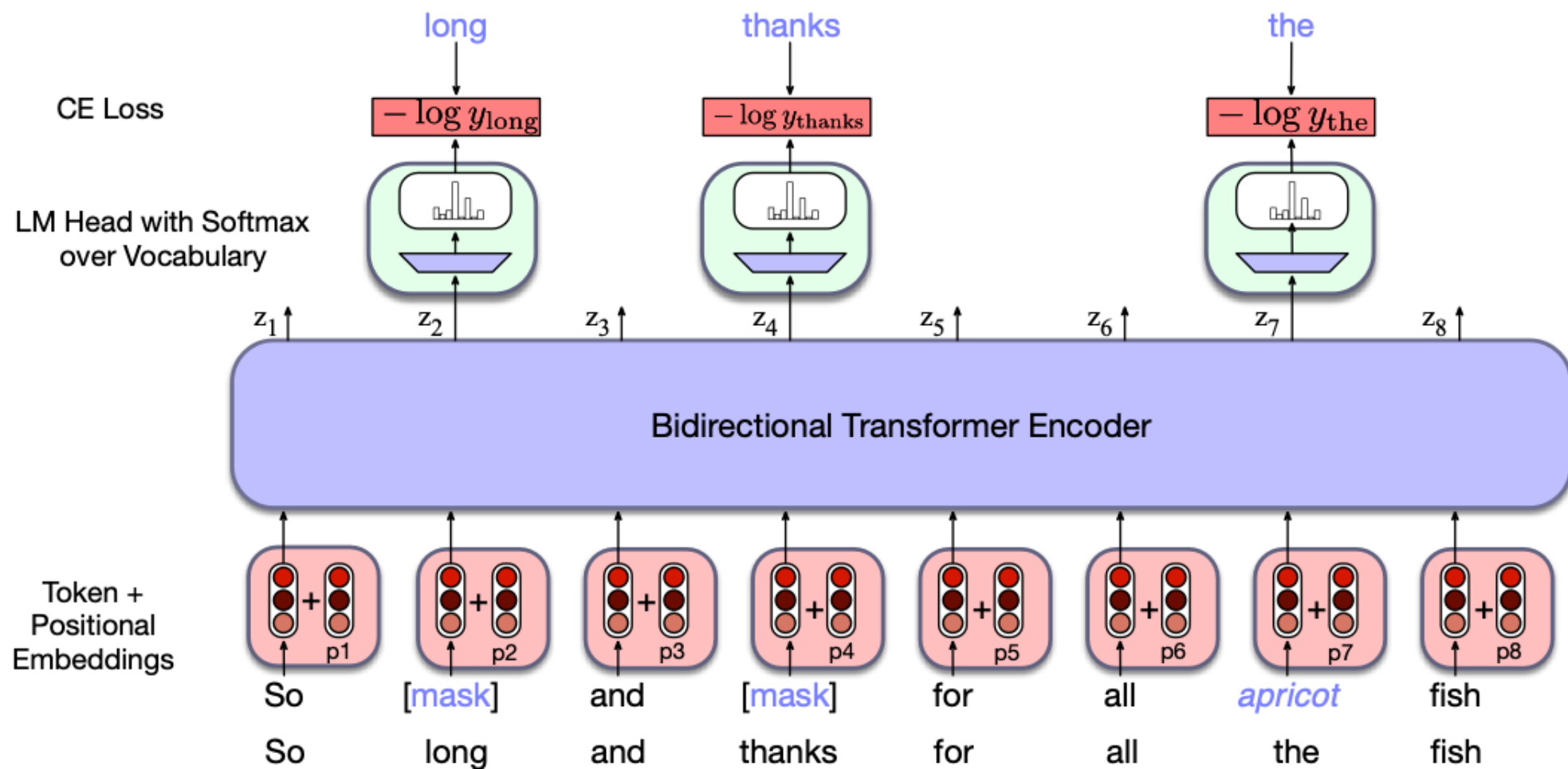
Ex., pour le x_i qui correspond à "long", la perte est la probabilité du token correct *long*, étant donné l'embedding caché $\mathbf{h}_i^L$):

$$\mathcal{L}_{\text{MLM}}(x_i) = -\log P(x_i \mid \mathbf{h}_i^L)$$

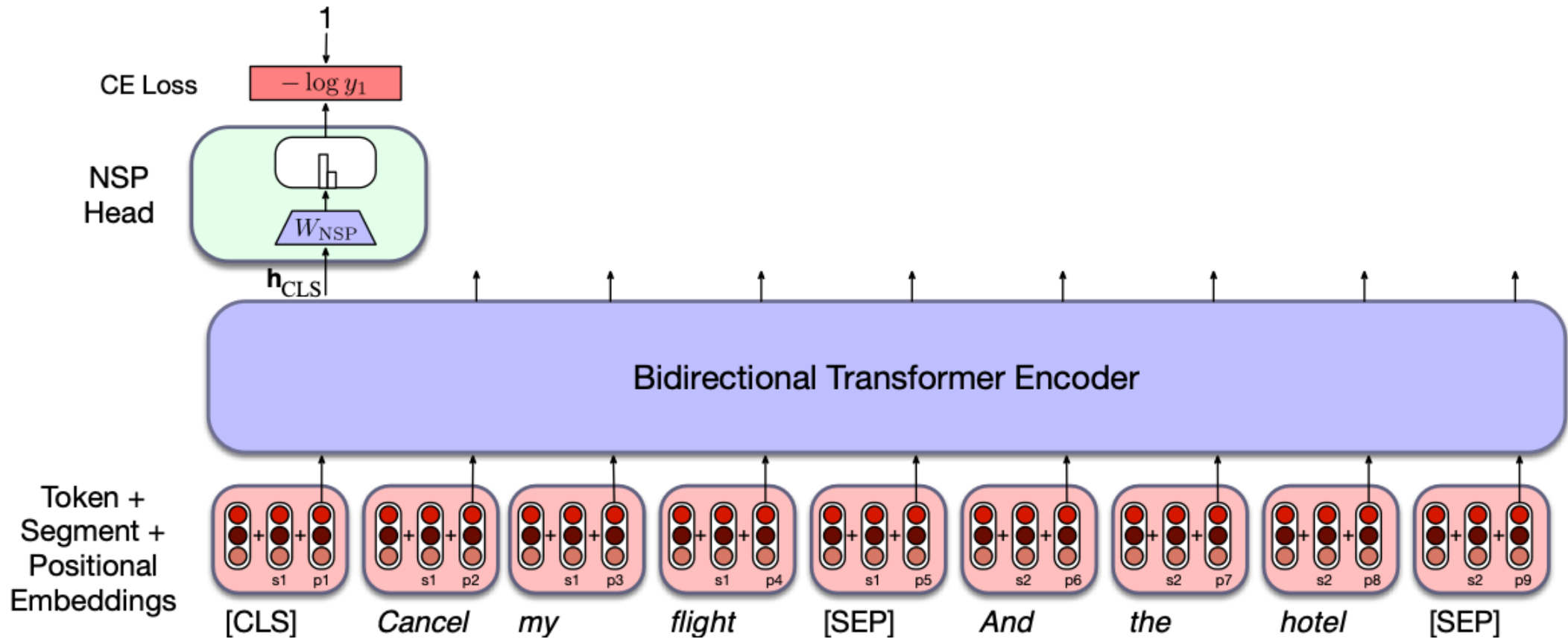
Nous obtenons les gradients en prenant la moyenne de cette perte sur la batch

$$\mathcal{L}_{\text{MLM}}(x_i) = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \log P(x_i \mid \mathbf{h}_i^L)$$

En détail



Perte NSP avec tête de classification



Perte de prédiction de la phrase suivante

Étant donné 2 phrases, le modèle prédit s'il s'agit d'une véritable paire de phrases adjacentes du corpus d'entraînement ou d'une paire de phrases non liées.

BERT présente deux tokens spéciaux

- [CLS] est au début de la paire de phrases d'entrée,
- [SEP] est placé entre les phrases, et aussi après la deuxième phrase

Et deux autres tokens spéciaux

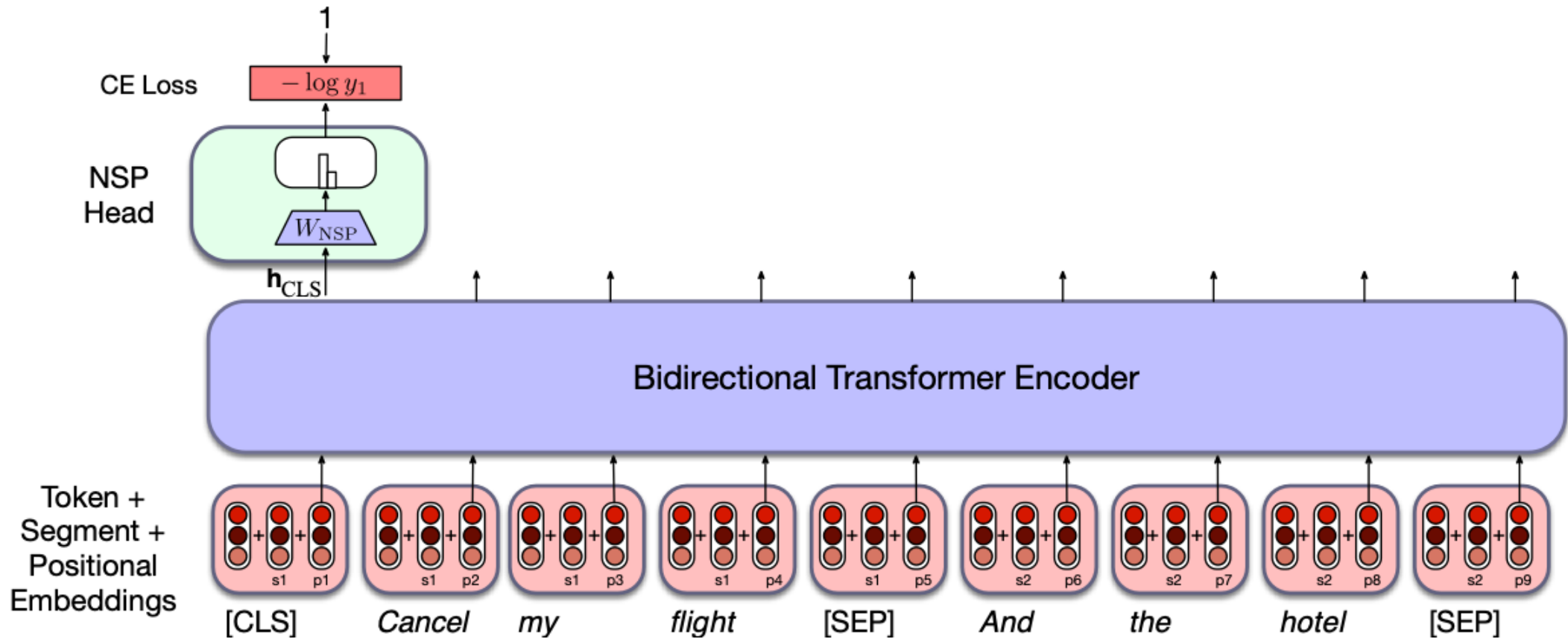
- [1st segment] et [2nd segment]

Ceux-ci sont un troisième embedding ajoutés aux embeddings d'entrée et positionnelle à chaque token

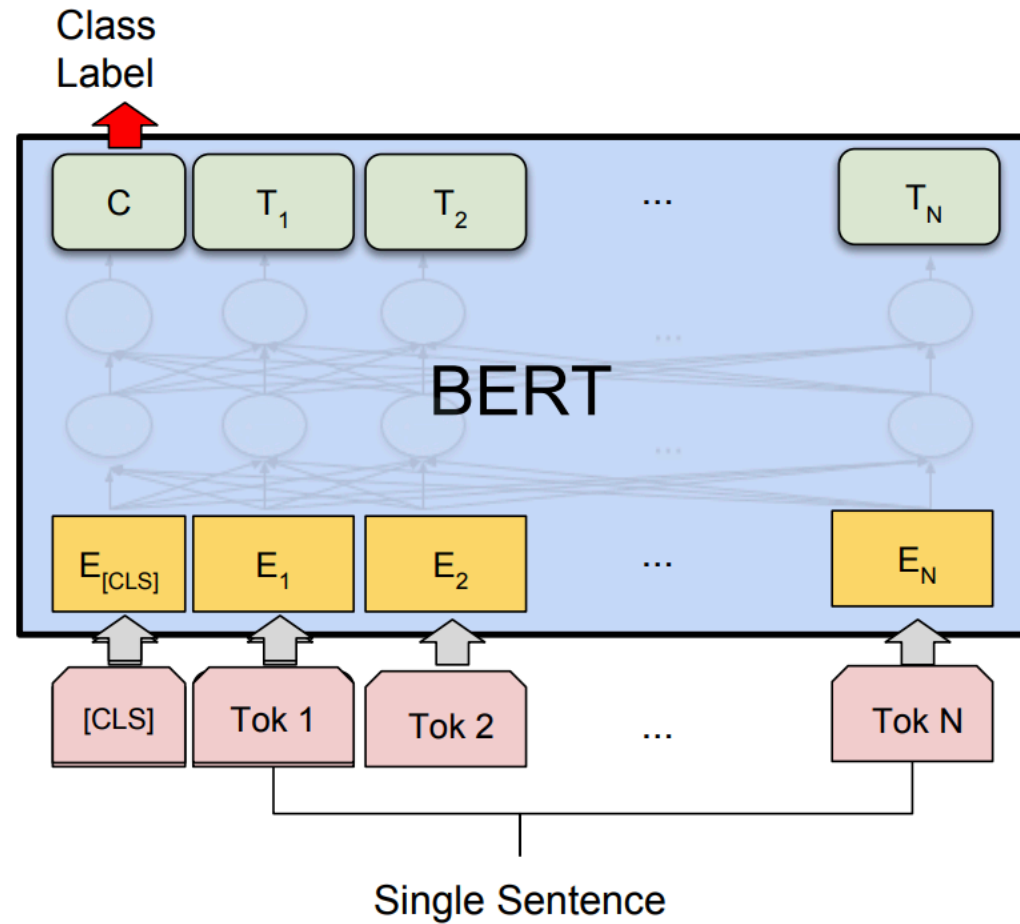
h_{CLS}^L à partir de la couche finale du token [CLS] est entré dans une tête de classification (poids W_{NSP}) qui prédit deux classes :

$$y_i = \text{softmax}(\mathbf{h}_{CLS}^L) \mathbf{W}_{NSP}$$

Perte NSP avec tête de classification



BERT pour la classification

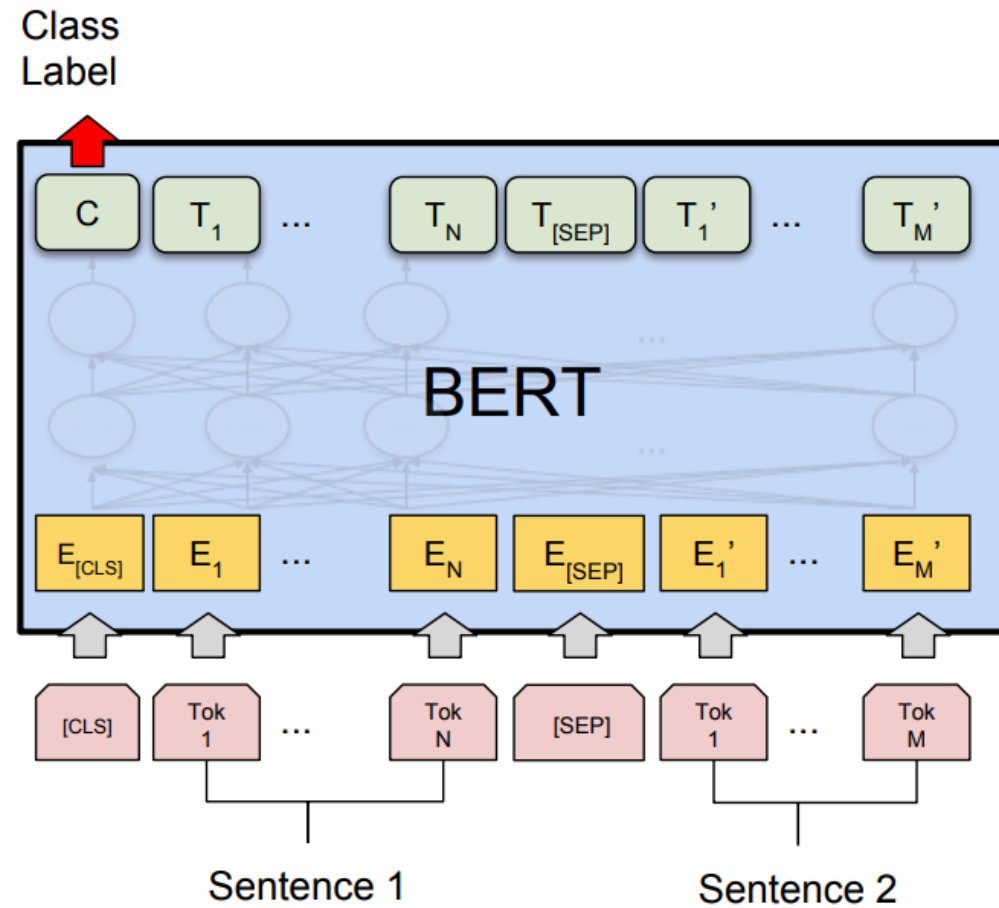


Classification de phrase

Attribuez une étiquette à une seule phrase :

- **Analyse du sentiment**
- **Jugements d'acceptabilité/grammaticaux**
(une phrase donnée est-elle grammaticale ?)
- **Détection du discours de haine** (une phrase donnée exprime-t-elle la haine ou encourage-t-elle la violence ?)

BERT pour la classification de paires de phrases



Classification paire de séquence

Attribuez une étiquette aux paires de phrases :

- **Détection de paraphrase** (Les deux phrases sont-elles paraphrases l'une de l'autre ?)
- **Implication logique** (La phrase A implique-t-elle logiquement la phrase B ?)
- **Cohérence du discours** (Dans quelle mesure la phrase B est-elle cohérente pour faire suite à la phrase A ?)

Exemple: Inférence du langage naturel

Les paires de phrases sont données à l'une des 3 étiquettes

- Neutral

a: Jon walked back to the town to the smithy.

b: Jon traveled back to his hometown.

- Contradicts

a: Tourist Information offices can be very helpful.

b: Tourist Information offices are never of any help.

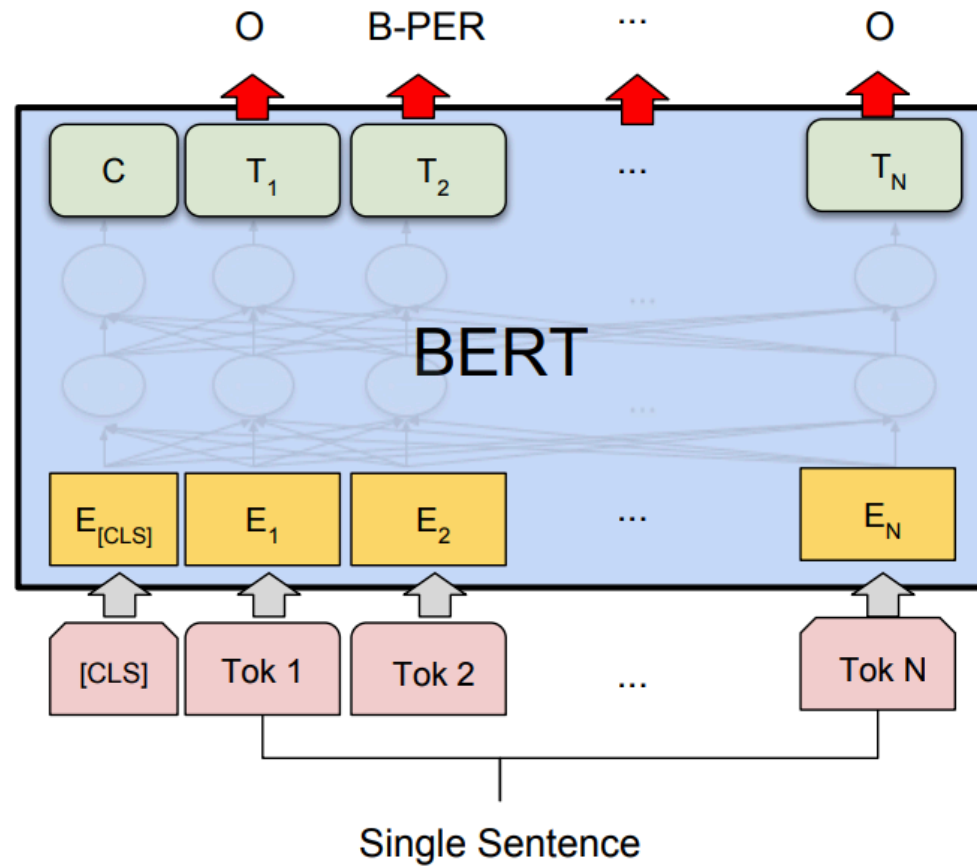
- Entails

a: I'm confused.

b: Not all of it is very clear to me.

Comment ? —> passer les paires prémisses/hypothèse à travers BERT et utiliser le vecteur de sortie pour le token [CLS] comme entrée à la tête de classification.

BERT pour l'étiquetage des séquences



BERT pour l'étiquetage des séquences

Attribuez une étiquette à partir d'un ensemble fixe d'étiquettes à chaque token de la séquence.

- Reconnaissance de l'entité nommée
- Marquage de partie du discours (POS-tagging)

Reconnaissance d'entité nommée

Une entité nommée est tout ce qui peut être appelé avec un nom propre : une personne, un lieu, une organisation

Reconnaissance d'entité nommée (NER) : trouver des séquences de texte qui constituent des noms propres et marquer le type de l'entité

Type	Tag	Sample Categories	Example sentences
People	PER	people, characters	Turing is a giant of computer science.
Organization	ORG	companies, sports teams	The IPCC warned about the cyclone.
Location	LOC	regions, mountains, seas	Mt. Sanitas is in Sunshine Canyon .
Geo-Political Entity	GPE	countries, states	Palo Alto is raising the fees for parking.

Reconnaissance d'entité nommée

Citing high fuel prices, [ORG United Airlines] said [TIME Friday] it has increased fares by [MONEY \$6] per round trip on flights to some cities also served by lower-cost carriers. [ORG American Airlines], a unit of [ORG AMR Corp.], immediately matched the move, spokesman [PER Tim Wagner] said. [ORG United], a unit of [ORG UAL Corp.], said the increase took effect [TIME Thursday] and applies to most routes where it competes against discount carriers, such as [LOC Chicago] to [LOC Dallas] and [LOC Denver] to [LOC San Francisco].

BIO Tagging

Ramshaw and Marcus (1995)

Une méthode qui nous permet de transformer une tâche de segmentation (trouver des limites d'entités) en une tâche de classification

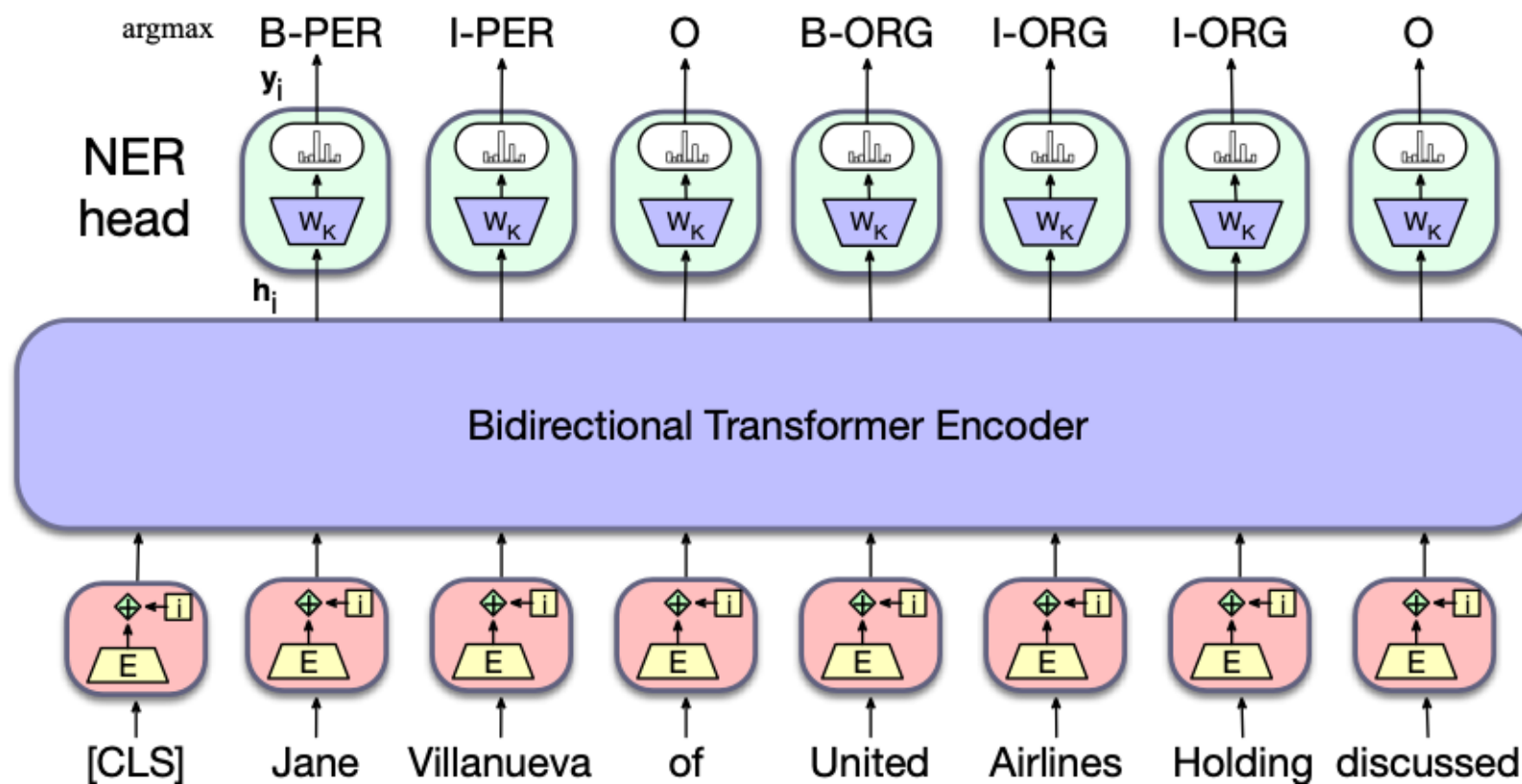
[**PER Jane Villanueva**] of [**ORG United**] , a unit of [**ORG United Airlines Holding**] , said the fare applies to the [**LOC Chicago**] route.

Words	IO Label	BIO Label	BIOES Label
Jane	I-PER	B-PER	B-PER
Villanueva	I-PER	I-PER	E-PER
of	O	O	O
United	I-ORG	B-ORG	B-ORG
Airlines	I-ORG	I-ORG	I-ORG
Holding	I-ORG	I-ORG	E-ORG
discussed	O	O	O
the	O	O	O
Chicago	I-LOC	B-LOC	S-LOC
route	O	O	O
.	O	O	O

Étiquetage de séquence

$$\mathbf{y}_i = \text{softmax}(\mathbf{h}_i^L \mathbf{W}_K)$$

$$\mathbf{t}_i = \text{argmax}_k(\mathbf{y}_i)$$



Recherche d'information

Un utilisateur a un besoin d'informations et a un ensemble de documents. Il veut trouver un document (ou des documents) pertinents dans l'ensemble pour satisfaire ses besoins.

Recherche d'information à grande échelle

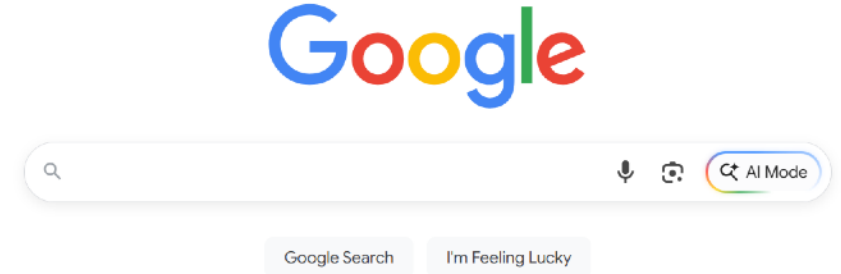


Google Search

I'm Feeling Lucky

Recherche d'information à grande échelle

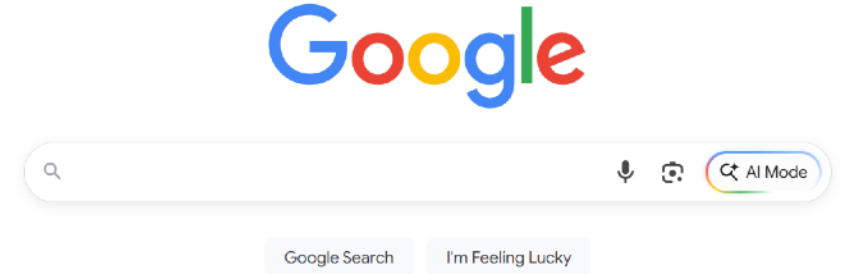
Recherche sur le web



Recherche d'information à grande échelle

Recherche sur le web

Recherche dans les courriels

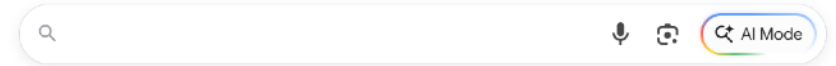


Recherche d'information à grande échelle

Recherche sur le web

Recherche dans les courriels

Recherche dans les documents d'entreprise



Google Search

I'm Feeling Lucky

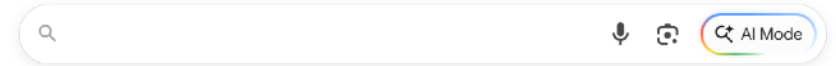
Recherche d'information à grande échelle

Recherche sur le web

Recherche dans les courriels

Recherche dans les documents d'entreprise

Recherche dans les dossiers médicaux personnels



Google Search

I'm Feeling Lucky

Recherche d'information à grande échelle

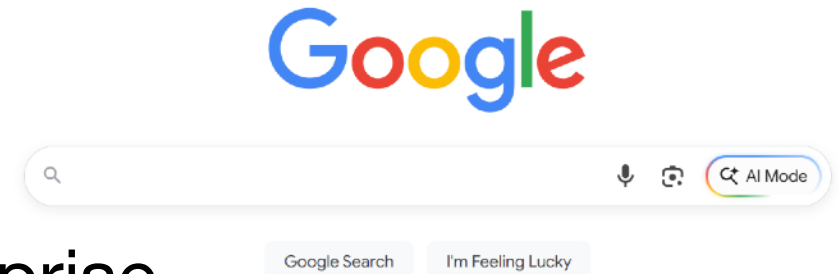
Recherche sur le web

Recherche dans les courriels

Recherche dans les documents d'entreprise

Recherche dans les dossiers médicaux personnels

Composante clé des LLMs dans la génération augmentée par la recherche (RAG)



Recherche ordonnée (ranked retrieval)

Le système retourne les k documents les plus pertinents

Classement par score de pertinence

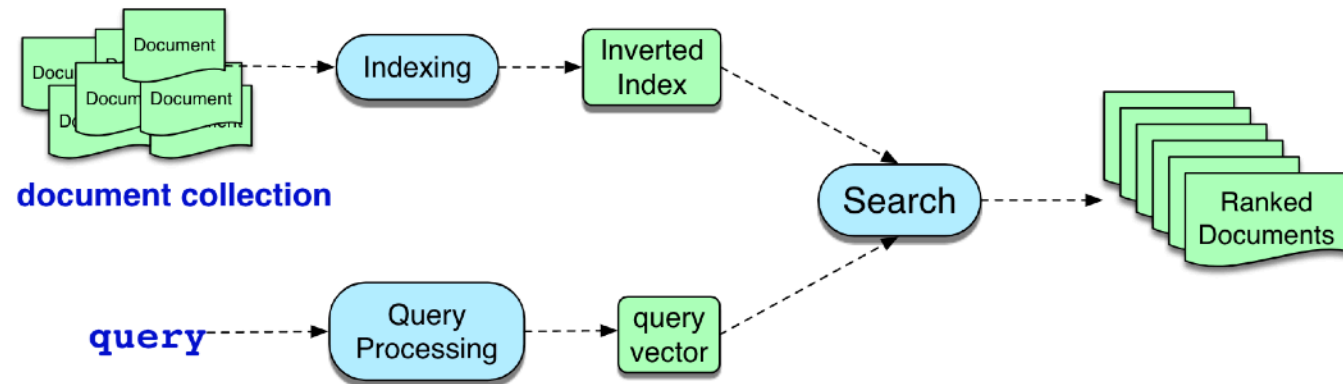
Objectif : estimer dans quelle mesure un document satisfait le besoin d'information.

Approximation par similarité textuelle entre la requête et le document

Recherche d'Information

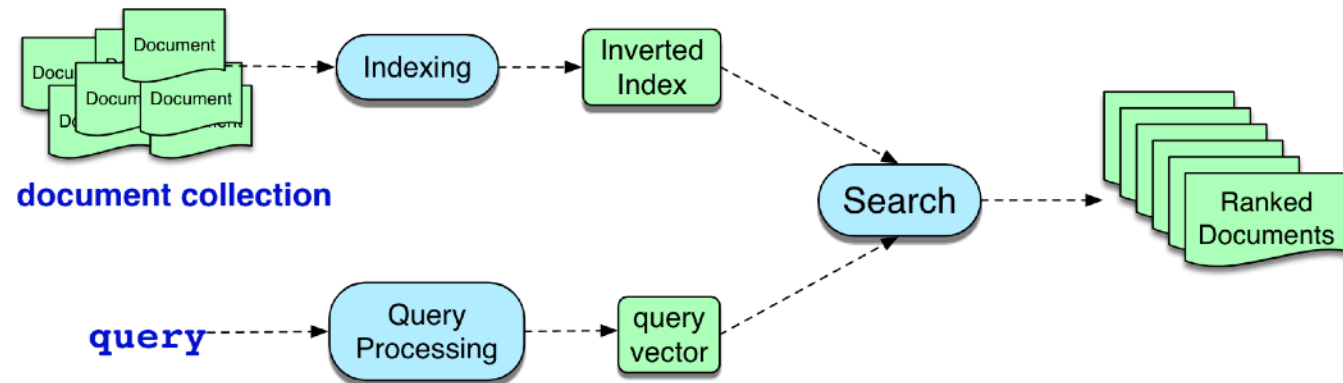
Récupérer des *documents* en fonction du *besoin d'information* de l'utilisateur

- Cas particulier : **recherche ad-hoc**
- Un utilisateur fournit une question à un système, qui renvoie ensuite un ou plusieurs documents d'un ensemble



Recherche d'information

- **Document:** L'unité de texte que le système IR indexe
- **Collection:** Ensemble de documents
- **Terme:** Un mot (ou un paragraphe) qui apparaît dans la collection
- **Requête:** Besoin d'information exprimé sous la forme d'un ensemble de termes



Deux architectures

Recherche épars

- Représenter la requête et le document en tant que vecteurs épars
- Pondéré par tf-idf, BM25

Recherche dense

- Utilisez LLM pour représenter la requête et le document en tant qu'embeddings denses

Dans les deux cas, la similitude est le produit scalaire ou le cosinus entre les représentations de requête et de document

Les types d'embeddings

- **Il existe trois principaux types d'embeddings utilisés en TALN et ils sont créés en utilisant trois méthodes différentes :**
 1. Embeddings épars et “Term Frequency-Inverse Document Frequency (TF-IDF)
 2. Embeddings denses et Word2Vec et l’algorithme skip-gram
 3. Embeddings contextualisés et les LLMs

Embeddings éparés et la notation avec tf-idf

- Valeur pondérée pour chaque terme

$$w_{t,d} = \text{tf}_{t,d} \times \text{idf}_f$$

- Comment notons-nous les requêtes et les documents ?
Similitude du cosinus

$$\text{score}(q, d) = \cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{|\mathbf{q}| |\mathbf{d}|}$$

Term frequency (tf) dans l'algorithme tf-idf

Nous pourrions imaginer utiliser le compte brut des mots dans un document:

$$\text{tf}_{t,d} = \text{count}(t,d)$$

Mais généralement nous écrasons un peu les comptes :

$$\text{tf}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t,d) & \text{if } \text{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

Inverse document frequency (idf)

$$\text{idf}_t = \log_{10} \left(\frac{N}{\text{df}_t} \right)$$

N est le nombre total de documents dans la collection

df_t est le nombre de document dans lesquels terme t est présent.

Notation avec tf-idf

$$\text{score}(q, d) = \cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{|\mathbf{q}| |\mathbf{d}|}$$

- Réécrivez le produit scalaire comme une somme de produits pour chaque terme dans la requête

$$\text{score}(q, d) = \sum_{t \in \mathbf{q}} \frac{\text{tf-idf}(t, q)}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i, q)}} \cdot \frac{\text{tf-idf}(t, d)}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i, d)}}$$

Notation avec tf-idf

$$\text{score}(q, d) = \sum_{t \in q} \frac{\text{tf-idf}(t, q)}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i, q)}} \cdot \frac{\text{tf-idf}(t, d)}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i, d)}}$$

Query: sweet love
Doc 1: Sweet sweet nurse! Love?
Doc 2: Sweet sorrow
Doc 3: How sweet is love?
Doc 4: Nurse!

Document 1						Document 2				
word	cnt	tf	tf-idf	n'lized	× q	cnt	tf	tf-idf	n'lized	× q
sweet	2	1.301	0.163	0.357	0.137	1	1.000	0.125	0.203	0.0779
nurse	1	1.000	0.301	0.661	0	0	0	0	0	0
love	1	1.000	0.301	0.661	0.610	0	0	0	0	0
how	0	0	0	0	0	0	0	0	0	0
sorrow	0	0	0	0	0	1	1.000	0.602	0.979	0
is	0	0	0	0	0	0	0	0	0	0
$ d_1 = \sqrt{.163^2 + .301^2 + .301^2} = .456$						$ d_2 = \sqrt{.125^2 + .602^2} = .615$				
Cosine: $\sum$ of column: 0.747						Cosine: $\sum$ of column: 0.0779				

BM25

$$\text{idf}_t = \log_{10} \left(\frac{N}{\text{df}_t} \right)$$

- Introduire les paramètres k et b

$$\text{score}(q, d) = \sum_{t \in q} \overbrace{\log \left(\frac{N}{\text{df}_t} \right)}^{\text{IDF}} \overbrace{\frac{tf_{t,d}}{k \left(1 - b + b \left(\frac{|d|}{|d_{\text{avg}}|} \right) \right) + tf_{t,d}}}^{\text{weighted tf}}$$

k (Saturation de fréquence des termes) : Contrôle combien la quantité d'occurrences supplémentaires d'un terme peut augmenter le score

b (Normalisation de la longueur du document) : Contrôle dans quelle mesure la longueur d'un document affecte le score, empêchant les documents longs d'être favorisés simplement parce qu'ils contiennent plus de mots

Recherche éparsse efficace

- Trouvez efficacement les documents qui contiennent des termes d'intérêt, en ignorant les documents qui ne contiennent aucun des mots de requête !
- **Index inversé**
 - Dictionnaire : liste des termes (+ document frequency {df})
 - Liste des publications : liste des identifiants de documents qui contiennent le terme (+ term frequency [tf] Dans chacun des documents)

Query: sweet love
Doc 1: Sweet sweet nurse! Love?
Doc 2: Sweet sorrow
Doc 3: How sweet is love?
Doc 4: Nurse!

how {1}	→ 3 [1]
is {1}	→ 3 [1]
love {2}	→ 1 [1] → 3 [1]
nurse {2}	→ 1 [1] → 4 [1]
sorry {1}	→ 2 [1]
sweet {3}	→ 1 [2] → 2 [1] → 3 [1]

Précision et rappel

- Les classificateurs heuristiques ont tendance à avoir une grande **précision** mais un très faible **rappel**.
- L'exactitude du classificateur est mesurée avec précision et rappel :

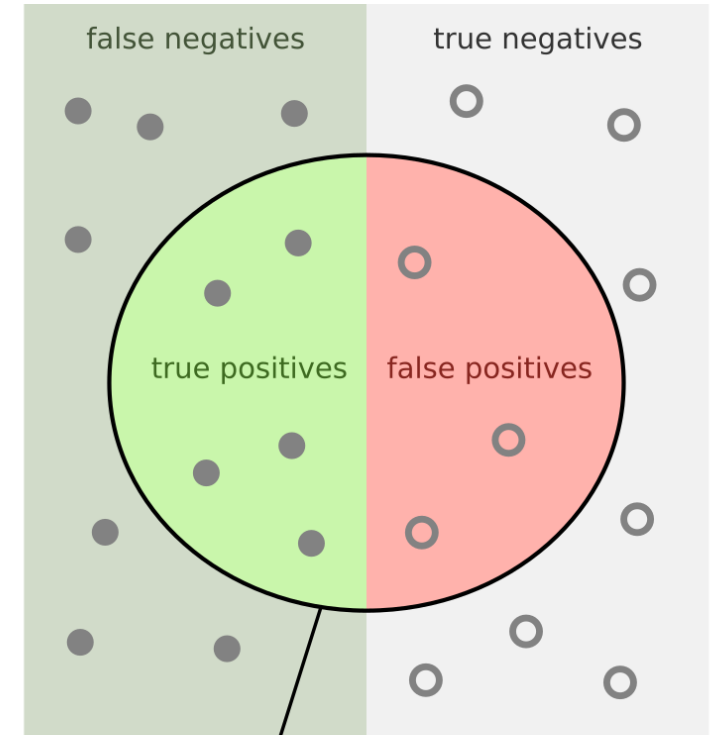
How many relevant items are retrieved?

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$


How many retrieved items are relevant?

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$


$$\text{Rappel} = \text{TP} / (\mathbf{FN} + \text{TP}) \quad \text{Précision} = \text{TP} / (\mathbf{FP} + \text{TP})$$



Précision et rappel en IR

- Chaque document est pertinent ou non
- **Précision** : Fraction des documents récupérés qui sont pertinents
- **Rappel** : Fraction de tous les documents pertinents qui sont récupérés
- Que T soit l'ensemble des documents retournés, R sont les documents pertinents dans T, N sont les documents non pertinents dans T, U sont tous les documents pertinents dans la collection

$$Precision = \frac{|R|}{|T|} \quad Recall = \frac{|R|}{|U|}$$

Évaluation des systèmes de recherche d'informations

- Nous nous soucions du rang du ou des documents cibles
- **Précision@k**: Fraction de documents pertinents vus à un rang particulier k
- **Rappel@k**: Fraction de documents pertinents trouvés à un rang particulier k

Rank	Judgment	Precision _{Rank}	Recall _{Rank}
1	R	1.0	.11
2	N	.50	.11
3	R	.66	.22
4	N	.50	.22
5	R	.60	.33
6	R	.66	.44
7	N	.57	.44
8	R	.63	.55
9	N	.55	.55
10	N	.50	.55

Évaluation des systèmes de recherche d'informations

- **Mean average precision (MAP)**
 - Itérer sur la liste classée de haut en bas
 - Notez la précision uniquement aux positions où un élément pertinent a été rencontré
 - Moyennez ces précisions sur l'ensemble de retour
 - Formellement: R_r est l'ensemble des documents pertinents au rang r ou au-dessus
- **Average precision:**
 - $\text{Precision}_r(d)$ Précision mesurée au rang où d a été trouvé

$$\text{AP} = \frac{1}{|R_r|} \sum_{d \in R_r} \text{Precision}_r(d)$$

Évaluation des systèmes de recherche d'informations

- **Mean average precision (MAP)**

$$AP = \frac{1}{|R_r|} \sum_{d \in R_r} \text{Precision}_r(d)$$

- **Average precision:**

- $\text{Precision}_r(d)$ Précision mesurée au rang où d a été trouvé
 - Étant donné un ensemble de requêtes Q

$$\text{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \text{AP}(q)$$

Évaluation des systèmes de recherche d'informations

- **Mean average precision (MAP)**
$$\text{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \text{AP}(q)$$

Rank	Judgment	Precision _{Rank}	Recall _{Rank}
1	R	1.0	.11
2	N	.50	.11
3	R	.66	.22
4	N	.50	.22
5	R	.60	.33
6	R	.66	.44
7	N	.57	.44
8	R	.63	.55
9	N	.55	.55
10	N	.50	.55

- Relevant docs at: 1, 3, 5, 6, 8
- Precisions: 1.0, 0.66, 0.60, 0.66, 0.63
- $\text{AP} = (1.0 + 0.66 + 0.60 + 0.66 + 0.63) / 5 = 3.55 / 5 = 0.71$

Problèmes avec l'IR épars

Le problème d'inadéquation du vocabulaire

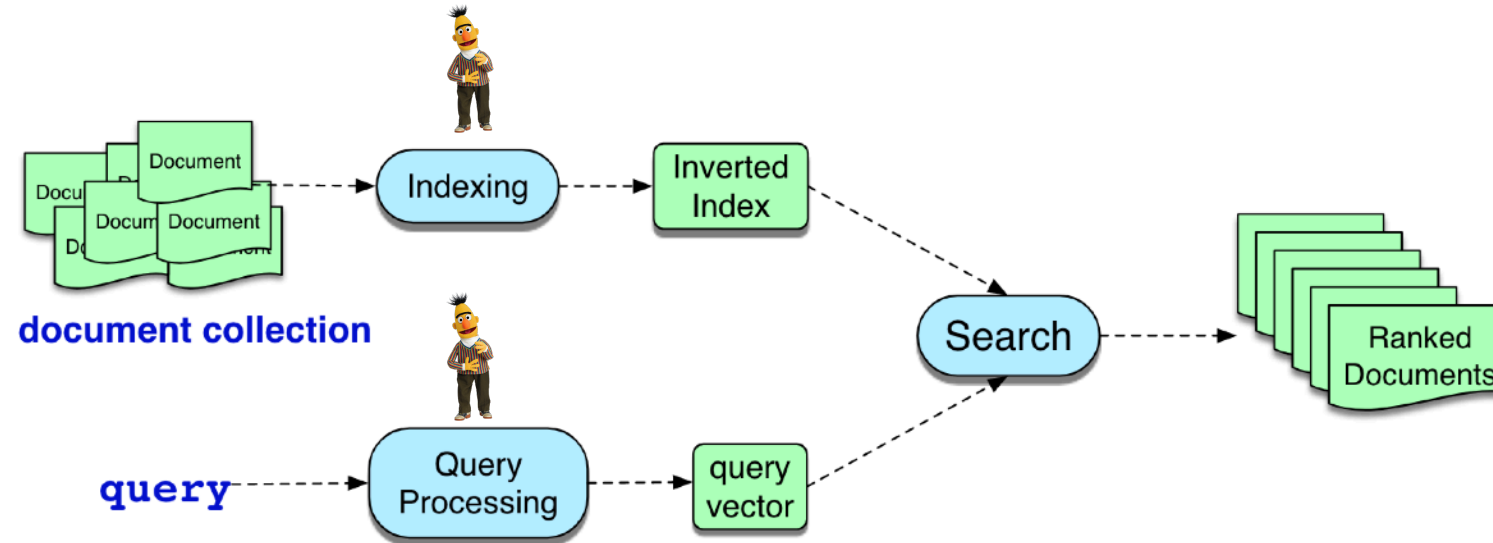
- Pour tf-idf ou BM25 Les similitudes cosinus ne fonctionnent que s'il y a un chevauchement exact de mots entre la requête et le document !
- Mais le rédacteur de requête ne peut pas connaître les mots exacts que le document pourrait inclure !

Recherche Dense

Au lieu de représenter des requêtes et des documents avec des vecteurs de comptage

Représentez les deux avec des embeddings !

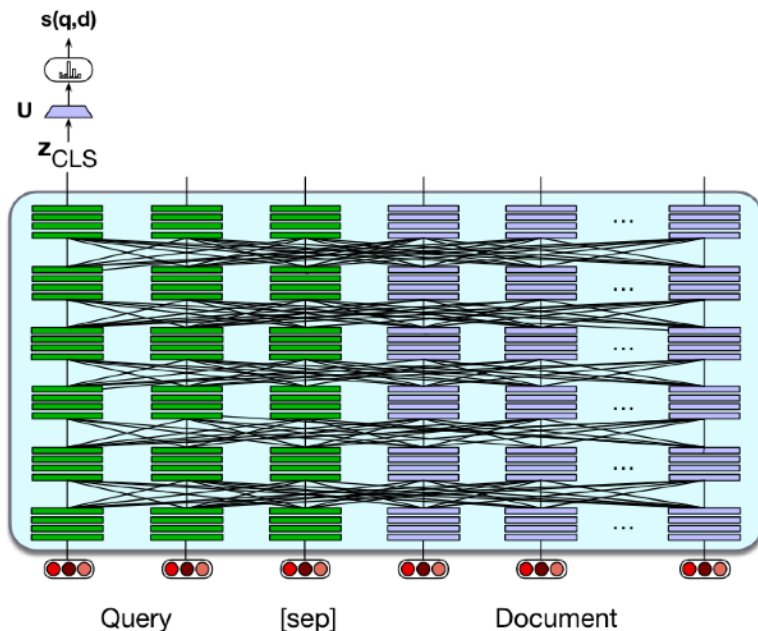
Recherche d'informations avec des LM pré-entraînés



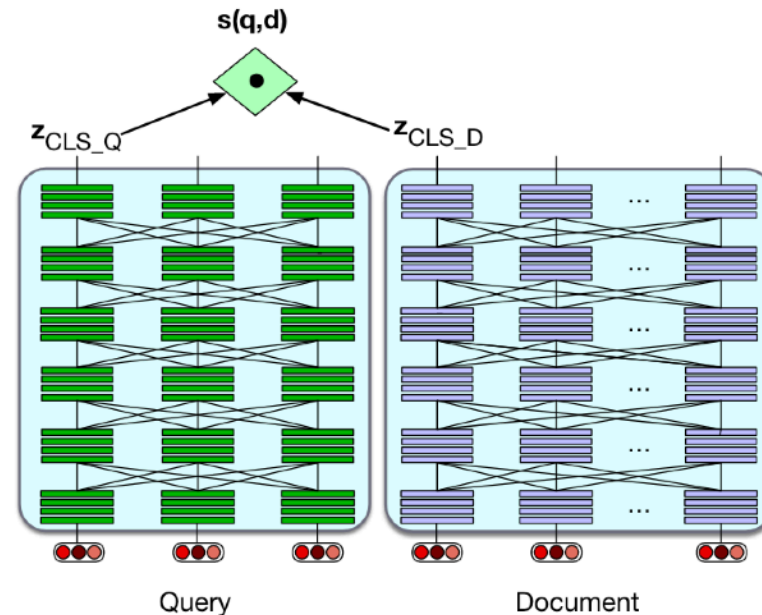
- Au lieu d'utiliser des vecteurs de compte de mots, utilisez des vecteurs denses à partir de modèles de langage pré-entraînés, par exemple, BERT

Recherche d'informations avec des LM pré-entraînés

- Deux façons potentielles d'indexer les requêtes et les documents
- Encoder conjointement la requête et chaque document
- Bi-encodeur : encoder la requête et les documents indépendamment



(a)



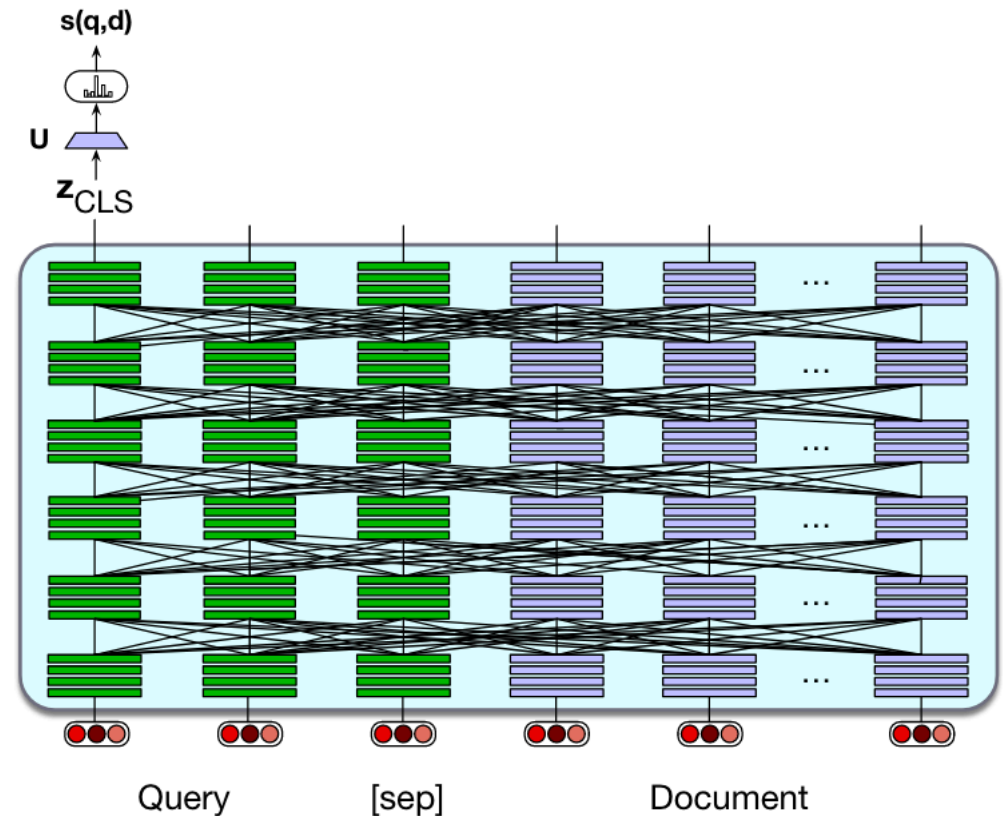
(b)

Recherche d'informations avec des LM pré-entraînés

- Encoder conjointement la requête et chaque document

$$\mathbf{z} = \text{BERT}(q; [\text{SEP}]; d)[\text{CLS}]$$

$$\text{score}(\mathbf{z}) = \text{softmax}(\mathbf{U}\mathbf{z})$$

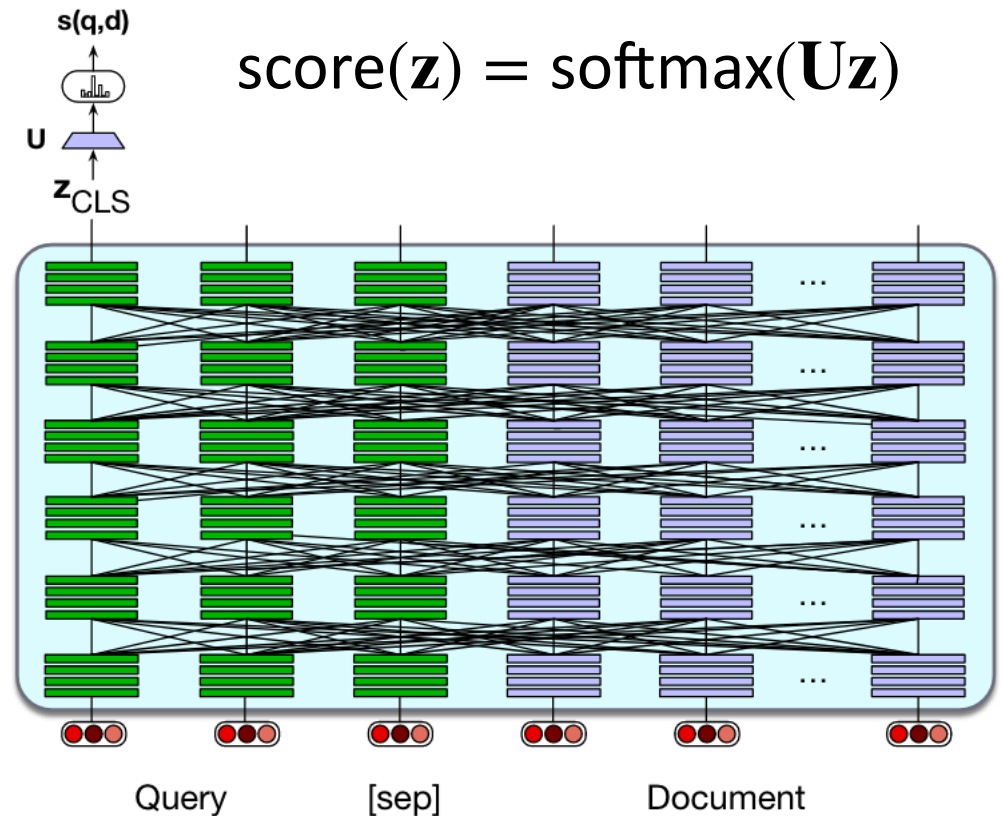


Recherche d'informations avec des LM pré-entraînés

- Cette approche est coûteuse
- Besoin de recoder chaque document pour chaque requête
- Cela devient rapidement peu pratique lorsqu'il s'agit de grandes collections de documents

$$\mathbf{z} = \text{BERT}(q; [\text{SEP}]; d)[\text{CLS}]$$

$$\text{score}(\mathbf{z}) = \text{softmax}(\mathbf{U}\mathbf{z})$$



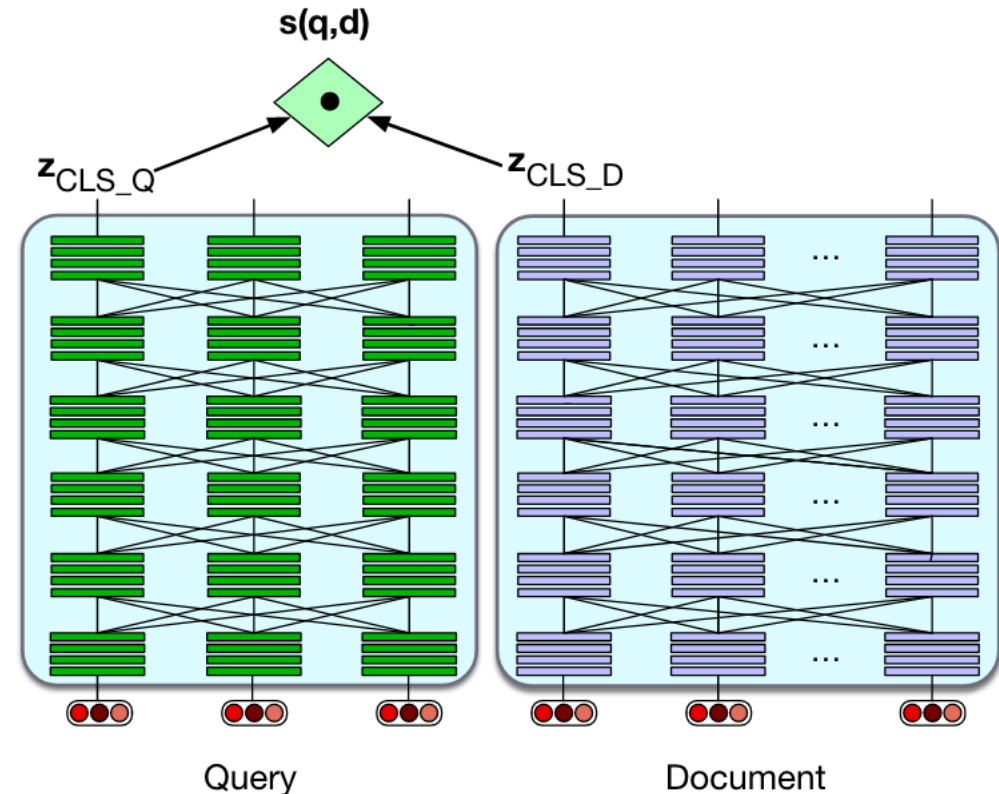
Recherche d'informations avec des LM pré-entraînés

- **Bi-encodeur:** Encoder des requêtes et des documents indépendamment

$$\mathbf{z}_q = \text{BERT}_{\text{query}}(q)[\text{CLS}]$$

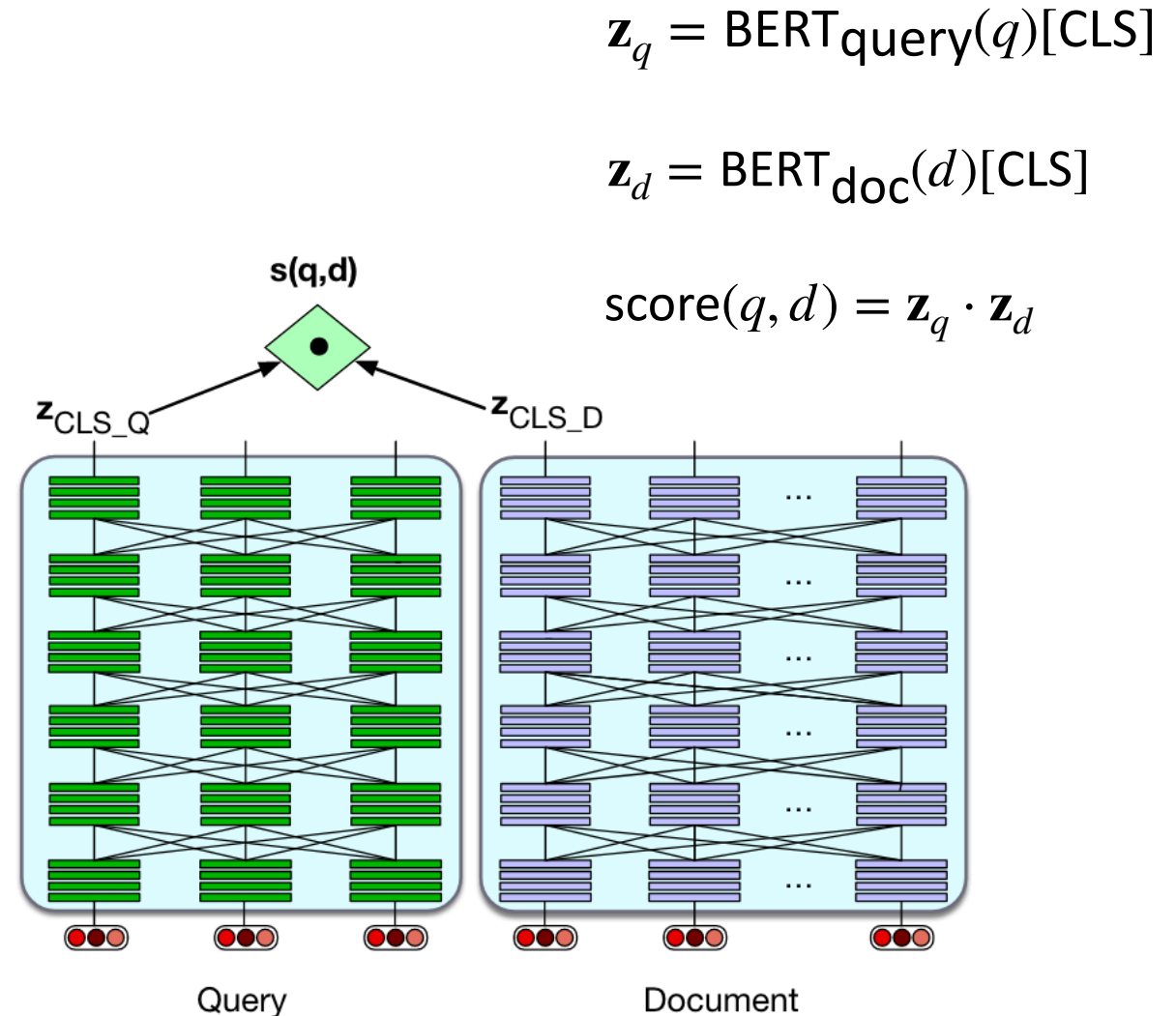
$$\mathbf{z}_d = \text{BERT}_{\text{doc}}(d)[\text{CLS}]$$

$$\text{score}(q, d) = \mathbf{z}_q \cdot \mathbf{z}_d$$



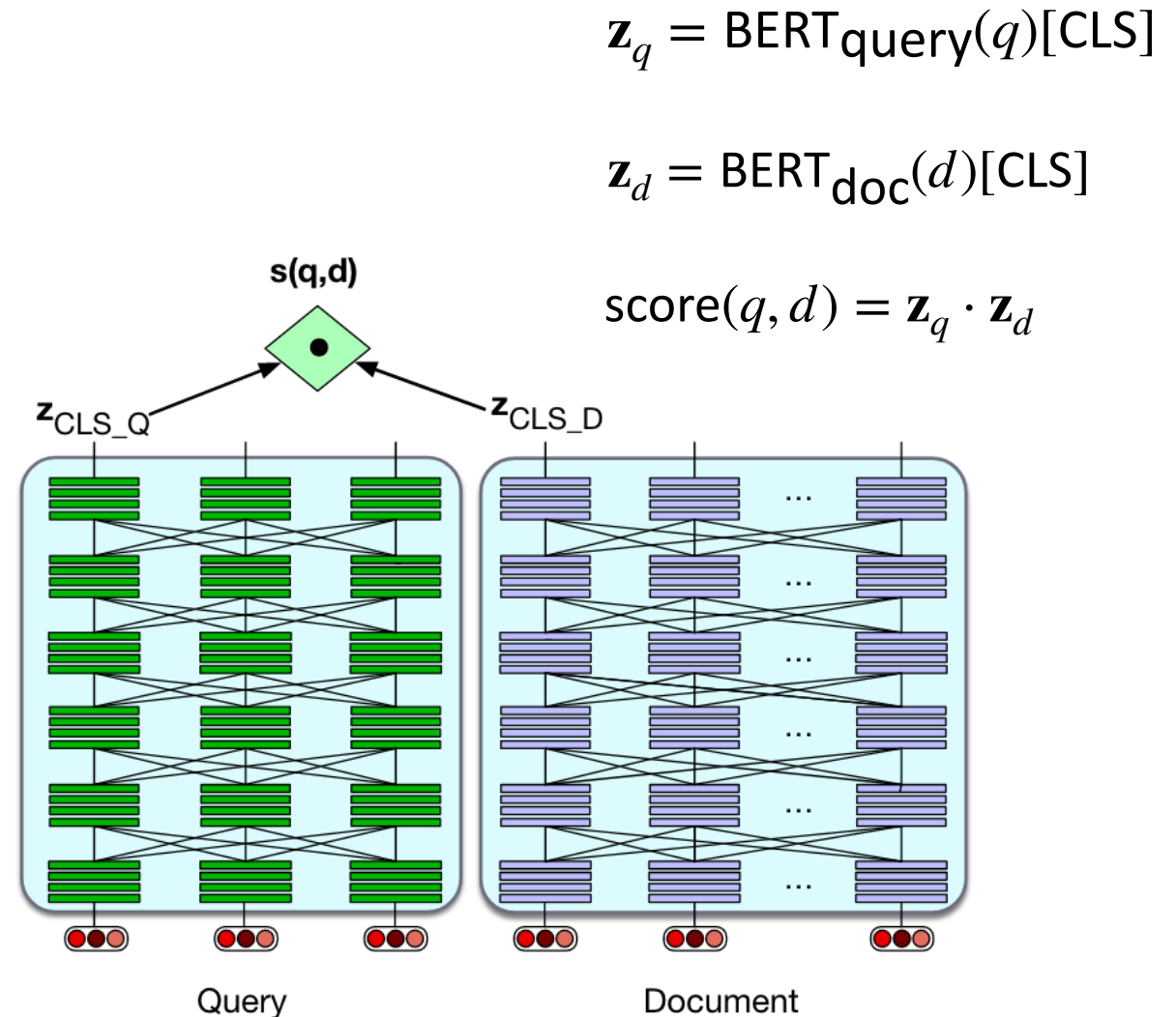
Recherche d'informations avec des LM pré-entraînés

- Les documents sont codés à l'avance
- Lorsqu'une nouvelle requête arrive, nous n'avons qu'à encoder cette requête
- Beaucoup plus efficace, mais a tendance à donner de pires résultats
- **Question** : Comment combiner les deux approches ?



Recherche d'informations avec des LM pré-entraînés

- Les documents sont codés à l'avance
- Lorsqu'une nouvelle requête arrive, nous n'avons qu'à encoder cette requête
- Beaucoup plus efficace, mais a tendance à donner de pires résultats
- **Question** : Comment combiner les deux approches ?
 - **Reclassement (reranking)**



Effacité de l'utilisation du reclassement : méthodes intermédiaires

1. Utilisez des méthodes efficaces (comme BM25) comme classement de la pertinence du premier passage pour chaque document
3. Ensuite, il suffit de reclasser les N meilleurs documents classés en utilisant des méthodes coûteuses comme la notation BERT simultanée.

Efficacité avec la recherche dense

Contrairement à la recherche éparses à l'index inversé, nous devons classer chaque document pour sa similitude avec la requête !

Efficacité avec la recherche dense:

- **Recherche des plus proches voisins (nearest neighbor search) :** trouver l'ensemble de vecteurs de documents denses qui ont le produit scalaire le plus élevé avec un vecteur de requête dense.
- Algorithme de recherche des plus proches voisins approximative comme **FAISS**(Johnson et al., 2017) sur GPUs
 - Approxime le vecteur doc par un vecteur quantifié plus petit

Retrieval-Augmented Generation (RAG)

La recherche d'informations joue un rôle central dans les LLM modernes.

Comment répondre à des questions factuelles telles que :

Où se trouve le musée du Louvre ?

Comment obtenir un script en latex ?

D'où vient l'énergie d'une explosion nucléaire ?

Demander/Prompt des informations à un LLM

✦ AI Overview

The main Louvre Museum is located in **Paris, France**, at the **Musée du Louvre, 75001 Paris, France**. It is situated on the Right Bank of the Seine River in the city's 1st arrondissement, housed within the historic Louvre Palace. [🔗](#)

- **Address:** Rue de Rivoli, 75001 Paris, France.

Demander/Prompt des informations à un LLM

Les LLM semblent garder des faits dans les connexions dans leurs couches feedforward !

✦ AI Overview

The main Louvre Museum is located in Paris, France, at the Musée du Louvre, 75001 Paris, France. It is situated on the Right Bank of the Seine River in the city's 1st arrondissement, housed within the historic Louvre Palace. [🔗](#)

- **Address:** Rue de Rivoli, 75001 Paris, France.

Problème : Les LLM hallucinent

Hallucination: Une réponse qui n'est pas fidèle aux faits du monde.

Dans le domaine juridique, il a été démontré que les LLM hallucinaient jusqu'à 88 % du temps !

Dahl et al. (2024)

Problème : ne peut pas utiliser les données propriétaires

Les gens doivent poser des questions sur :

- E-mail personnel.
- Demandes de soins de santé aux dossiers médicaux.
- Documents d'entreprise internes
- Découverte de documents juridiques

Problème : Ne peut pas gérer les données dynamiques

Les LLM ne peuvent pas répondre aux questions sur l'évolution rapide des informations

Les choses qui se sont passées la semaine dernière

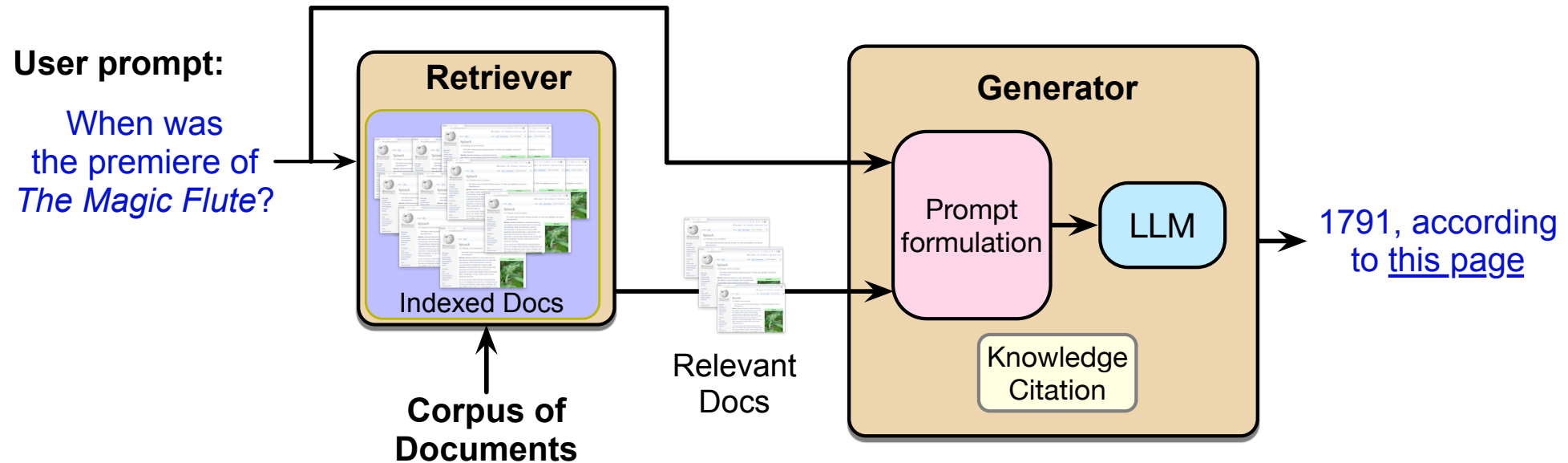
En général, les données changent au fil du temps

Solution: RAG

Retrieval-Augmented Generation

1. Utilisez IR pour **récupérer (retrieve)** des documents d'une collection
2. Ensuite, utilisez LLM pour **générer** une réponse conditionnée sur les documents

Retrieval Augmented Generation (RAG)



RAG de base

Étant donné une collection de documents D et une requête d'utilisateur q :

Appeler un retriever pour trouver les top k passages

Créer un prompt qui inclut q et les passages trouvés

Appeler un LLM avec la prompt

Schematic of a RAG Prompt

retrieved passage 1

retrieved passage 2

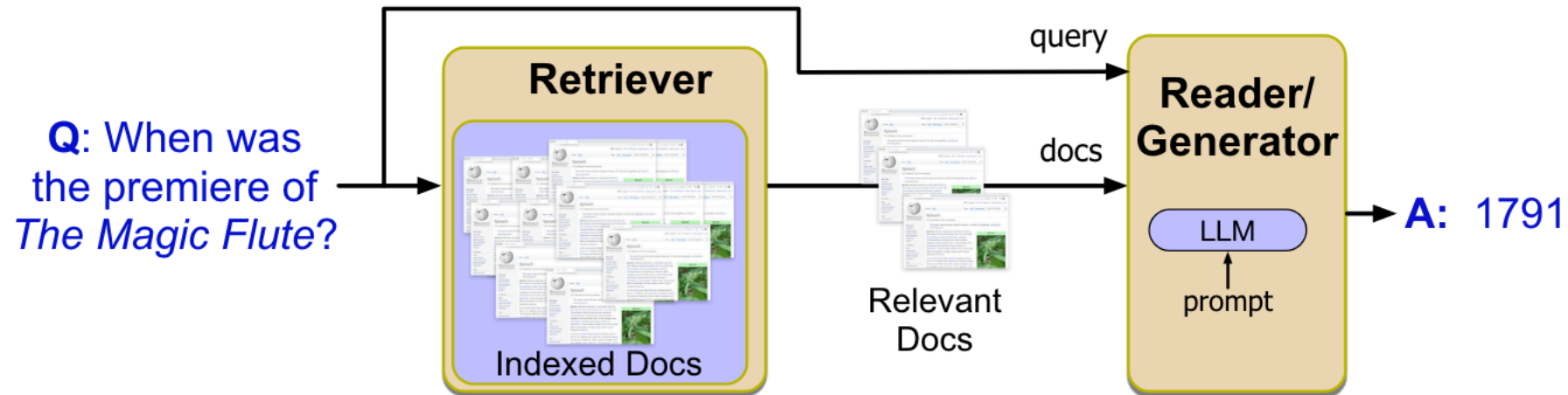
...

retrieved passage k

Based on these texts, answer this question: What year was the premiere of *The Magic Flute*?

Recherche d'informations avec des LM pré-entraînés

- Questions-réponses (QA) en tant que problème de recherche d'information
- Trouver des pièces justificatives pour une requête
- Générer une réponse basée sur les meilleurs documents



Améliorer la recherche dense grâce au peaufinement

- Considérez le document à droite
- Il existe de nombreuses requêtes pour lesquelles ce document pourrait être pertinent
 - Quand HEC a-t-elle été fondée ?
 - Puis-je faire un doctorat à HEC ?
 - Qu'est-ce que Viger Square ?
 - Etc.
- Idéalement, tout cela doit être encodé dans la représentation du document

HEC Montréal ([French](#): *Hautes études commerciales de Montréal*; [English](#): *High Commercial Studies of Montreal*) is a [bilingual](#) public business school located in [Montreal](#), [Quebec](#), Canada. Founded in 1907, HEC Montréal is the graduate business school of the [Université de Montréal](#) and is the first established school of management in Canada.^{[2][3]}

HEC Montréal offers undergraduate, graduate, and post-graduate programs, including Bachelor of Business Administration (BBA), Master of Science in Administration (MSc), Master of Management (MM), Master of Business Administration (MBA), and PhD in Administration, in addition to a joint Executive MBA program with [McGill University](#).

HEC Montréal was founded in 1907 by the [Board of Trade of Metropolitan Montreal](#). Its initial building in [Viger Square](#) is now called the [Gilles Hocquart Building](#).^[2]

In 1988, a group of HEC students established Jeux du Commerce, where more than 1300 students from 14 universities in Eastern Canada gather annually for academic, social, and sports events.^{[4][5]} A similar competition has been established in [Western Canada](#) called [JDC West](#).

As of 2021, the centenary of the HEC Montréal Alumni Association, the school had over 100,000 alumni.^[6]

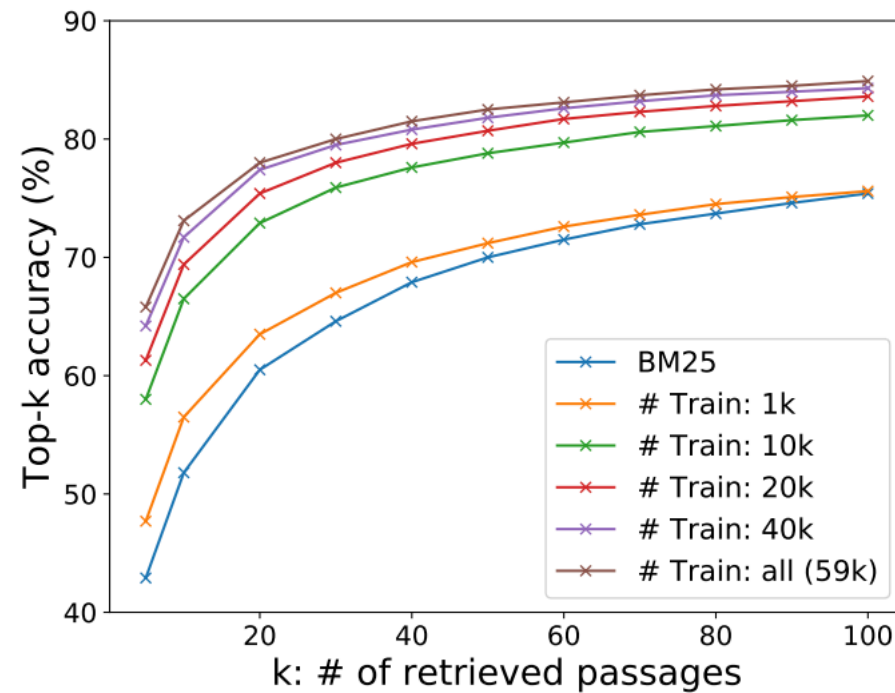
Améliorer la recherche dense grâce au peaufinement

- Utilise la collection $\{(q_i, d_i^+, d_i^-)\}_{i=1}^n$ de paires de requêtes avec des documents positifs and négatifs
- Apprendre une définition de pertinence implicite basée sur cette collection via l'**apprentissage contrastif**
- Intuition: Déplacer les embeddings des documents positifs plus proche de celui de la requête, et pousser ceux des documents négatifs plus loin

$$\mathcal{L}(q_i, d_i^+, \{d_{i,j}^-\}_{j=1}^k) = \frac{\exp(\mathbf{q} \cdot \mathbf{d}_i^+)}{\exp(\mathbf{q}_i \cdot \mathbf{d}_i^+) + \sum_{j=1}^k \exp(\mathbf{q}_i \cdot \mathbf{d}_{i,j}^-)}$$

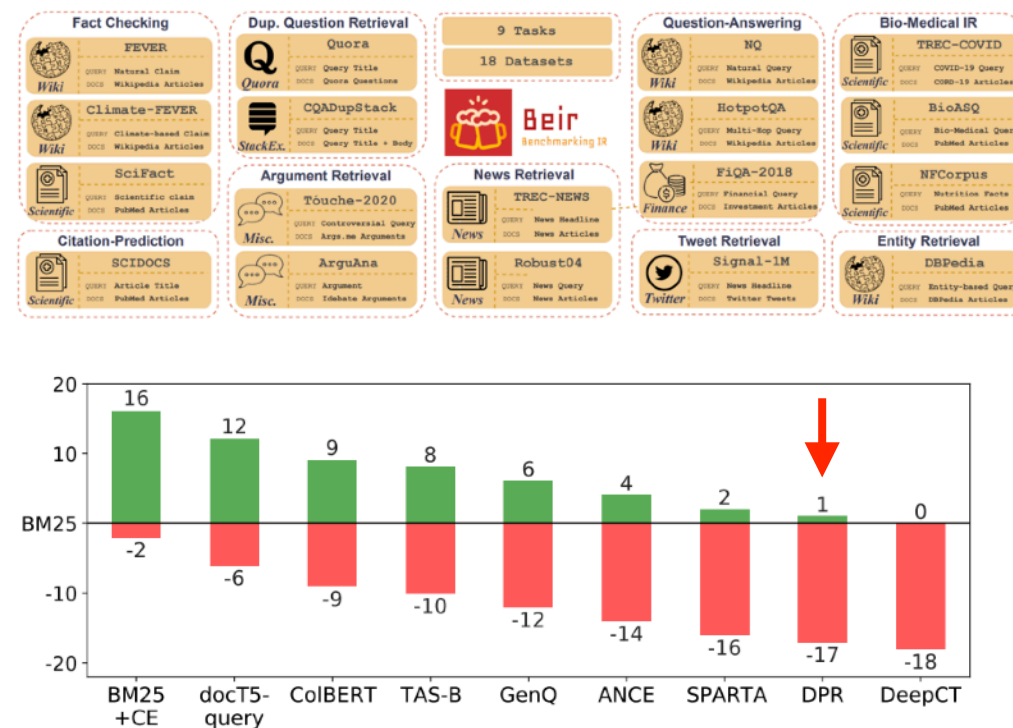
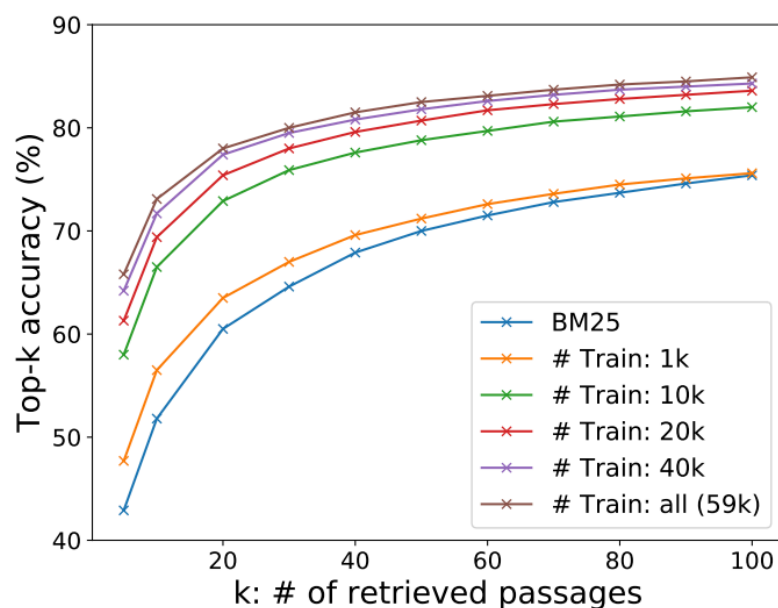
Recherche dense de passage pour QA à domaine ouvert

- Comment se compare-t-elle à BM25 ?



Recherche dense de passage pour QA à domaine ouvert

- Que se passe-t-il en dehors de la distribution d'entraînement ?
- Quelle pourrait être la raison ?



[pause de 15 minutes]

Étude de cas : Un compagnon IA d'assurance

Votre client est une compagnie d'assurance. L'évaluation équitable des réclamations demande beaucoup de formation et de connaissances accumulées, mais l'entreprise fait face à beaucoup de roulement d'employés (l'assurance n'est pas toujours le plus heureux des emplois !) Ce qui signifie que ces connaissances continuent de se perdre et que la formation des nouveaux employés est coûteuse. Comme solution pour réduire le temps de transfert de connaissances, vous proposez de créer un assistant d'IA qui aidera les agents à réclamation avec leur évaluation initiale. Son rôle sera d'identifier (1) le type de réclamation le plus probable et (2) les sections du contrat d'assurance du client qui font référence à ce type de réclamation. Il devrait ensuite (3) générer une évaluation initiale de l'admissibilité de la réclamation et des résolutions possibles. Vous avez accès aux documents suivants : (i) les réclamations écrites des clients et leurs contrat d'assurance complets (déjà mappés), (ii) l'ensemble de toutes les réclamations résolues et leurs résolutions des 5 dernières années, (iii) une liste des catégories de réclamations.

Étude de cas : Un compagnon IA d'assurance

- Esquissez la structure procédurale de votre assistant d'IA, en décrivant chaque étape et les technologies de TALN qui seront nécessaires. Si plusieurs sont possibles (par exemple, différents LM pourraient être utilisés), mettez en évidence lequel choisiriez-vous et pourquoi ?
- Identifiez les limites de vos solutions et où elles peuvent introduire des erreurs. Quel peut être votre niveau de responsabilité et d'imputabilité pour ces erreurs ?
- Considérez les défis possibles que votre solution peut présenter en matière de souveraineté des données, de confidentialité des clients et de coûts de calcul pour l'entreprise.